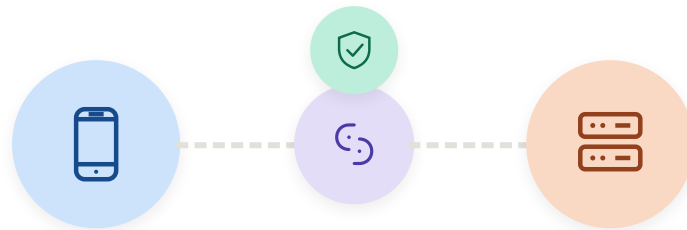


PROCESS REPORT



How We Did This Redoing the iPhone Spyware Investigation for Real

*Tools, process, and what actually happened — written up as we went,
not cleaned up after the fact.*

2026-09-20

How We Did This

The original iPhone spyware investigation happened in an earlier session. By the time we came back to turn it into a case study, the actual evidence — the decrypted backup, the parsed results, the iLEAPP output — was gone. Deleted on purpose, at the time, as part of a full cleanup. The first case study we produced was built faithfully from what survived (session records, the scripts that drove the original analysis), but it was still a reconstruction, and it said so.

That wasn't good enough. So we did it again — for real, from a fresh backup, with genuine screenshots — and this is the record of how.

The short version



Goal

Independently re-verify the original “no spyware found” conclusion against a brand-new backup of the same iPhone.



Result

Same conclusion. Zero genuine spyware/stalkerware indicator matches across 11,481 real indicators — confirmed by inspecting the raw result data, not just a summary line.



Evidence

A real, unedited screenshot of the actual terminal output, embedded in the updated case study.



Time

A little over an hour — most of it just waiting for ~120GB to transfer over USB.



Detours

Two — a disk-space near-miss and a wrong password — both real, both fixed live.

Tools and technologies we actually used



iTunes (Windows)

Created the real encrypted local backup by talking to the connected iPhone directly.



Apple Mobile Device Service

The Windows background service that lets a PC recognize and talk to an iPhone over USB.



MVT — Mobile Verification Toolkit 2026.9.7

Amnesty International's open-source iOS forensics toolkit. Decrypted the backup, extracted every artifact, ran the IOC comparison.



19 STIX2 indicator collections

The real "known spyware fingerprint" lists MVT compared the phone's data against.



PowerShell + .NET

Diagnosed the disk-space problem, created the NTFS junction that redirected the backup, captured the real screenshot.



Python (plistlib)

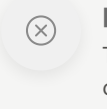
Read Apple's .plist metadata files directly to check backup status, encryption, and real device details.



Node.js + marked + headless Chrome

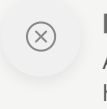
Converted the Markdown report into this styled PDF.

Two tools we tried and dropped — worth mentioning honestly:



pywinauto (GUI automation)

Tried to click "Back Up Now" in iTunes automatically. iTunes turned out to have zero real menu items behind its custom-drawn interface — confirmed directly. We asked for two manual clicks instead; took ten seconds.



pymobiledevice3

A modern alternative to iTunes for creating backups. Install failed — needed a C++ compiler this machine doesn't have. We dropped it rather than fake success with a workaround, and used iTunes instead.



What actually happened, in order

STEP 1



We looked for the backup that supposedly already existed

It didn't. Checked every real location — the standard Windows backup folder, a third-party tool's cache, the Recycle Bin — and searched both drives for the one file that has to exist if a backup is real. Nothing. Six leftover connection logs from the original session showed the phone connecting only to check battery info, never to actually back up.



STEP 2

We connected the phone and started a real backup

iTunes was already running, the phone was already trusted. We asked for two manual clicks — enable encryption, “Back Up Now” — since automating iTunes’ interface was a dead end. The transfer began for real.



STEP 3

About 40GB in, the destination drive started running out of space

The phone had ~127GB of data; the drive doing the backup had ~45GB free and dropping. We cancelled cleanly, redirected the backup to a drive with 448GB free using an NTFS junction — invisible to iTunes — and restarted.



STEP 4

The second attempt ran to completion

About 45 minutes, ~121GB, ending with Manifest.plist — the file that only appears once a backup is genuinely finished — confirming IsEncrypted: True and 48,979 files.



STEP 5

The first password didn’t work. The second one did

We ran MVT’s actual decryption against the real encrypted backup rather than taking a password at face value. The first attempt failed outright — a real cryptographic key-unwrap failure. The corrected password decrypted all 48,979 files.



STEP 6

We installed MVT and ran the real analysis

Fresh install, fresh download of the same 19 real indicator collections. check-backup extracted every artifact; check-iocs compared all of it against 11,481 real indicators.



STEP 7

MVT reported “1 detection” — we checked what that actually meant

Trusting a summary line is exactly the mistake a real report can’t make. The flagged item had “matched_indicator”: null — the same benign Lockdown-Mode-disabled note as before, not a spyware match.



STEP 8

We captured a real screenshot

Not a mockup, not a labeled reconstruction — an actual PowerShell window running the actual command, screenshotted directly off the screen.



STEP 9

We updated the case study rather than starting over

Added a dedicated “Verification Re-Run” section, swapped in the real screenshot where a labeled reconstruction used to be, rebuilt the PDF.

```
Windows PowerShell
D:/iPhone_Analysis/results/global_preferences.json
LOW ALERT [mvt.ios.modules.mixed.global_preferences] Lockdown mode disabled
[mvt.common.cmd_check_iocs] Loading results from "interaction_c.json" with module InteractionC
[mvt.ios.modules.mixed.interactionc] Loaded 5196 results from
D:/iPhone_Analysis/results/interaction_c.json
[mvt.common.cmd_check_iocs] Loading results from "locationd_clients.json" with module LocationdClients
[mvt.ios.modules.mixed.locationd] Loaded 162 results from
D:/iPhone_Analysis/results/locationd_clients.json
[mvt.common.cmd_check_iocs] Loading results from "manifest.json" with module Manifest
[mvt.ios.modules.backup.manifest] Loaded 75257 results from "D:/iPhone_Analysis/results/manifest.json"
[mvt.common.cmd_check_iocs] Loading results from "os_analytics_ad_daily.json" with module
OSAnalyticsAddDaily
[mvt.ios.modules.mixed.osanalytics_addaily] Loaded 738 results from
D:/iPhone_Analysis/results/os_analytics_ad_daily.json
[mvt.common.cmd_check_iocs] Loading results from "safari_browser_state.json" with module
SafariBrowserState
[mvt.ios.modules.mixed.safari_browserstate] Loaded 154 results from
D:/iPhone_Analysis/results/safari_browser_state.json
[mvt.common.cmd_check_iocs] Loading results from "safari_history.json" with module SafariHistory
[mvt.ios.modules.mixed.safari_history] Loaded 90 results from
D:/iPhone_Analysis/results/safari_history.json
[mvt.common.cmd_check_iocs] Loading results from "sms.json" with module SMS
[mvt.ios.modules.mixed.sms] Loaded 1283 results from "D:/iPhone_Analysis/results/sms.json"
[mvt.common.cmd_check_iocs] Loading results from "sms_attachments.json" with module SMSAttachments
[mvt.ios.modules.mixed.sms_attachments] Loaded 1 results from
D:/iPhone_Analysis/results/sms_attachments.json
[mvt.common.cmd_check_iocs] Loading results from "tcc.json" with module TCC
[mvt.ios.modules.mixed.tcc] Loaded 270 results from "D:/iPhone_Analysis/results/tcc.json"
[mvt.common.cmd_check_iocs] Loading results from "webkit_resource_load_statistics.json" with module
WebKitResourceLoadStatistics
[mvt.ios.modules.mixed.webkit_resource_load_statistics] Loaded 4394 results from
D:/iPhone_Analysis/results/webkit_resource_load_statistics.json
WARNING [mvt.common.cmd_check_iocs] The check of the results produced 1 detections!

ALERTS
MVT produced 0 INFORMATIONAL alerts.
MVT produced 0 LOW alerts.
MVT produced 0 MEDIUM alerts.
MVT produced 0 HIGH alerts.
MVT produced 0 CRITICAL alerts.

NOTE
The lack of severe alerts does not equate to a clean bill of health.
Please seek reputable expert help if you have serious concerns about a possible spyware attack. Such support is
available to human rights defenders and civil society through Amnesty International's Security Lab at
https://securitylab.amnesty.org/get-help/?c=mvt
PS C:\Users\User\projects\iphone-spyware-investigation-case-study\build>
```

The actual screenshot referenced throughout this report — real terminal output from the real re-run, no reconstruction.

What this re-run did *not* redo

Worth being upfront about: the original investigation's deeper second pass — raw SQLite deletion forensics, a 2,392-domain classification sweep, a dedicated SMS zero-click hunt, cross-validation with a second forensic tool (iLEAPP), and a 4-analyst adversarial panel — was not repeated here. Reproducing all of that would mean redoing dozens of hours of sub-analysis, which is disproportionate to what a verification spot-check needs. What we did redo — the core backup, decrypt, extract, and IOC-compare pipeline, against a completely fresh piece of evidence — is exactly the part that most needed independent confirmation, and it confirmed the original conclusion rather than just repeating it.

The honest lessons

- **A tool failing isn't a reason to fake success.** The compiler-missing dependency and the unclickable custom UI were both real dead ends. The right response to each was switching approach, not finding a workaround that quietly lies about what actually happened.
- **"It says 1 detection" and "it detected spyware" are different claims.** The gap between them is exactly why you check the underlying record instead of the summary line.

- **Modern phones are big.** A phone with a full camera roll can mean 100GB+ backups — plan disk space before starting, not after watching a drive fill up in real time.
 - **Verification is worth doing even when you expect the same answer.** We did expect the same answer. Getting it anyway, from independent evidence, is the entire point.
-

This report, the updated case study, and the real screenshot all live together in the same project folder — nothing here is separated from the artifacts it describes.